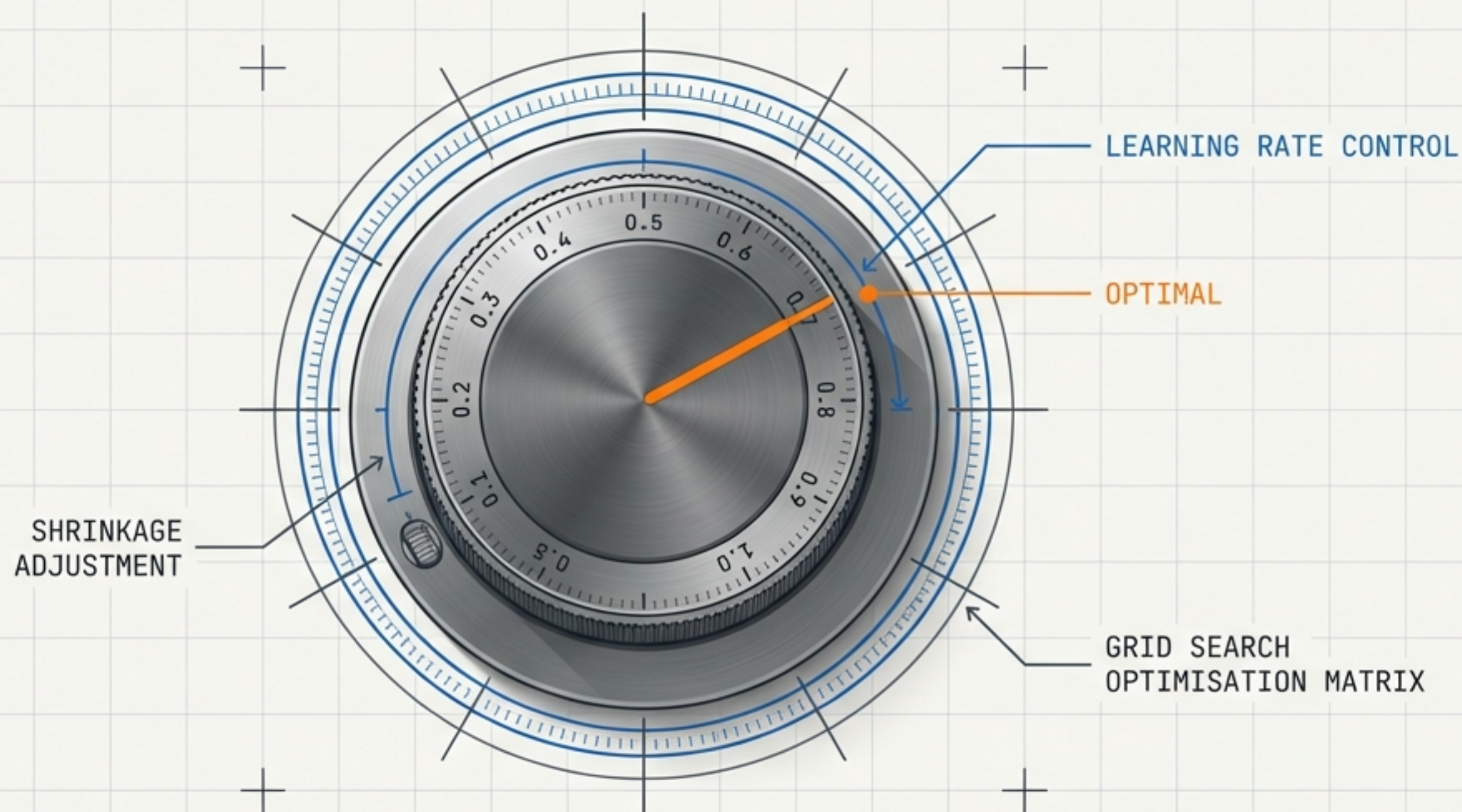


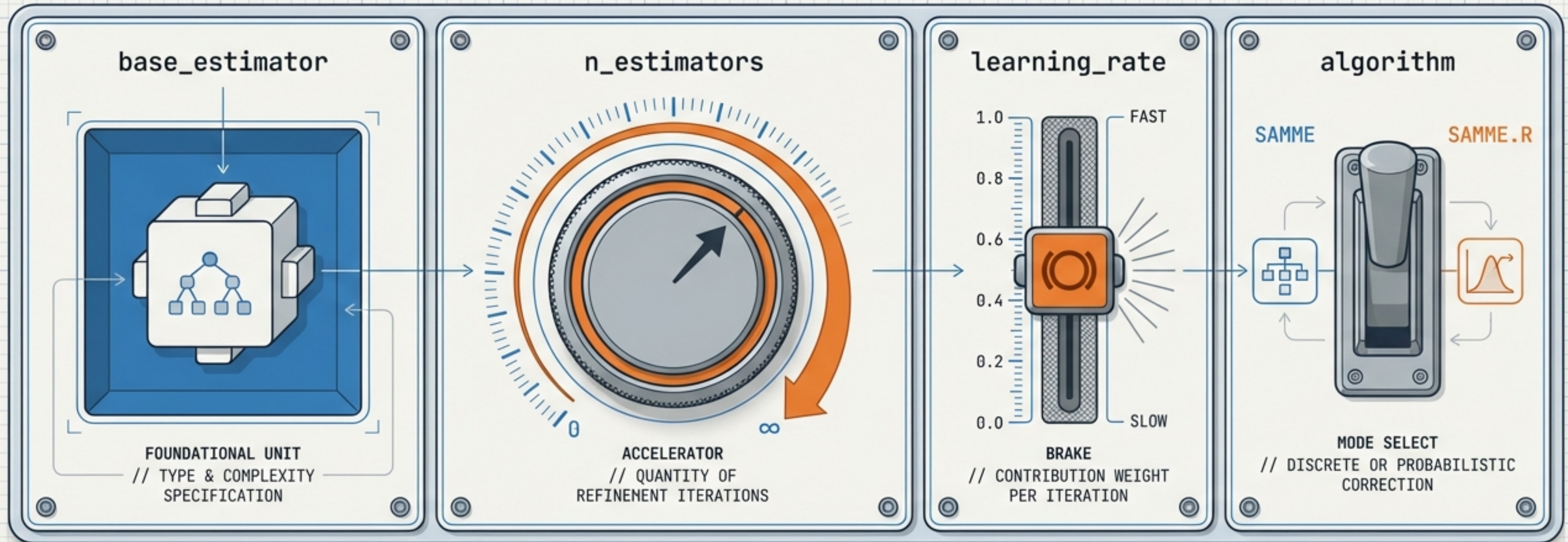
TUNING THE ADABOOST ENGINE

A visual guide to hyperparameter mechanics, the **shrinkage trade-off**, and **grid** optimisation.



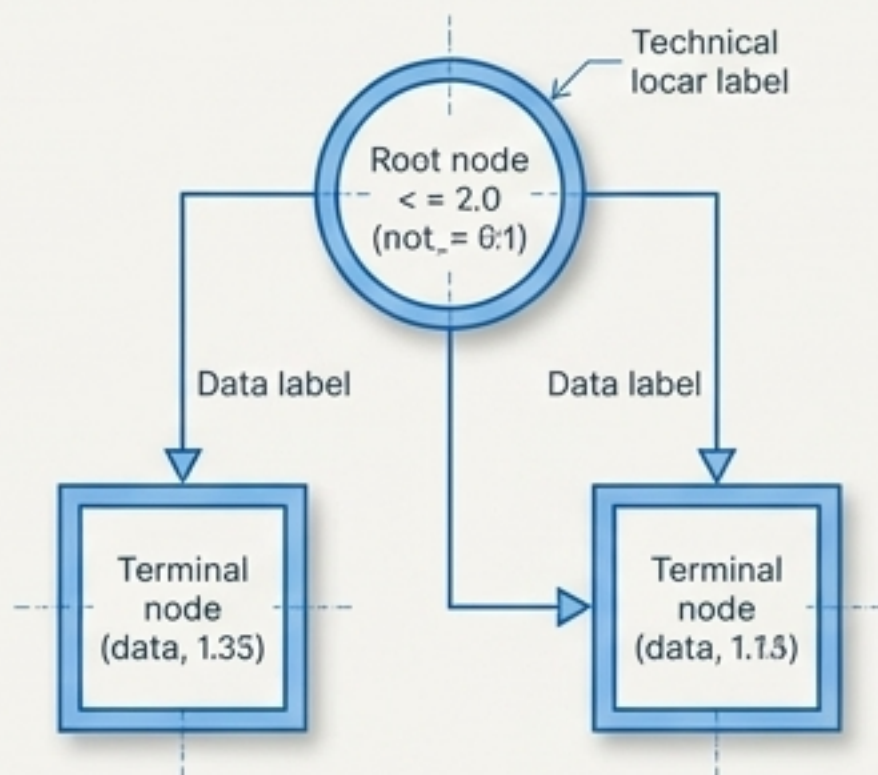
The Four Control Dials of AdaBoost

Unlike heavily parameterised algorithms, AdaBoost relies on a streamlined architecture. Mastery requires tuning just four mechanical levers. The true power lies in understanding how the accelerator interacts with the brake.



The Foundation: base_estimator

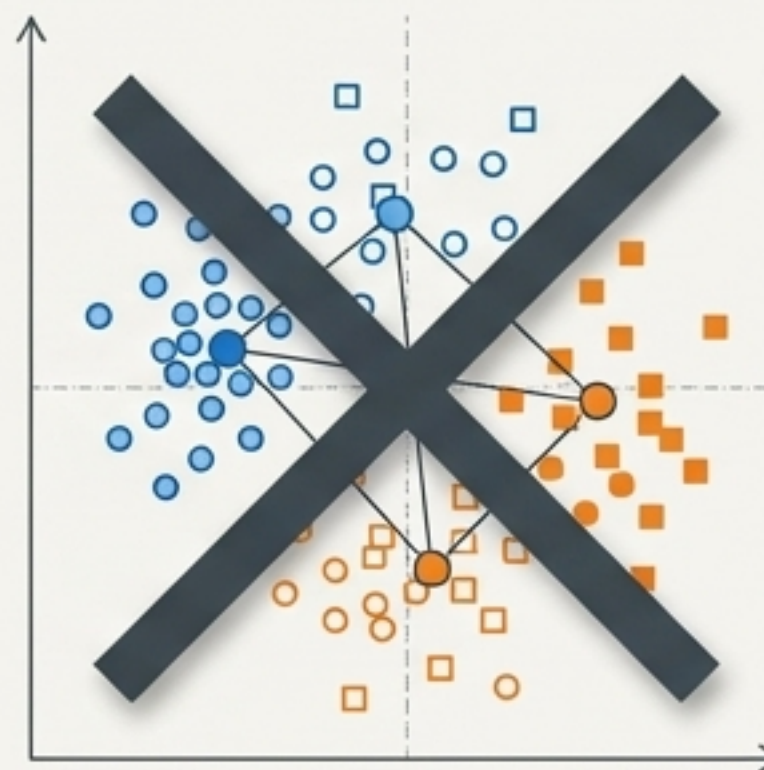
The Decision Stump



A Decision Tree constrained to `max_depth=1`. It provides a deliberate weak learning signal.

The 99.9% Rule: While other models are permitted, practical application dictates using the default Decision Stump for optimal performance.

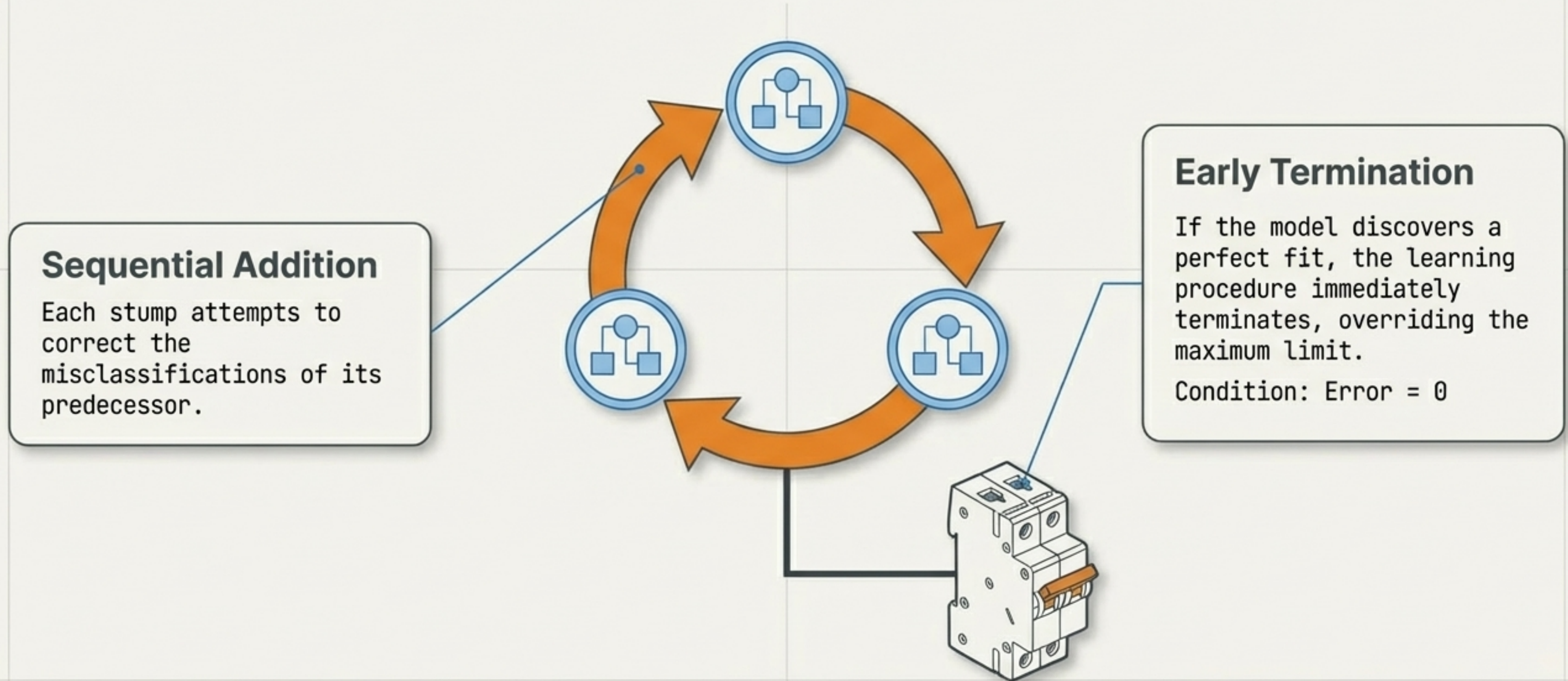
Why not KNN?



Lacks support for sample weighting, a fundamental requirement for AdaBoost's sequential learning.

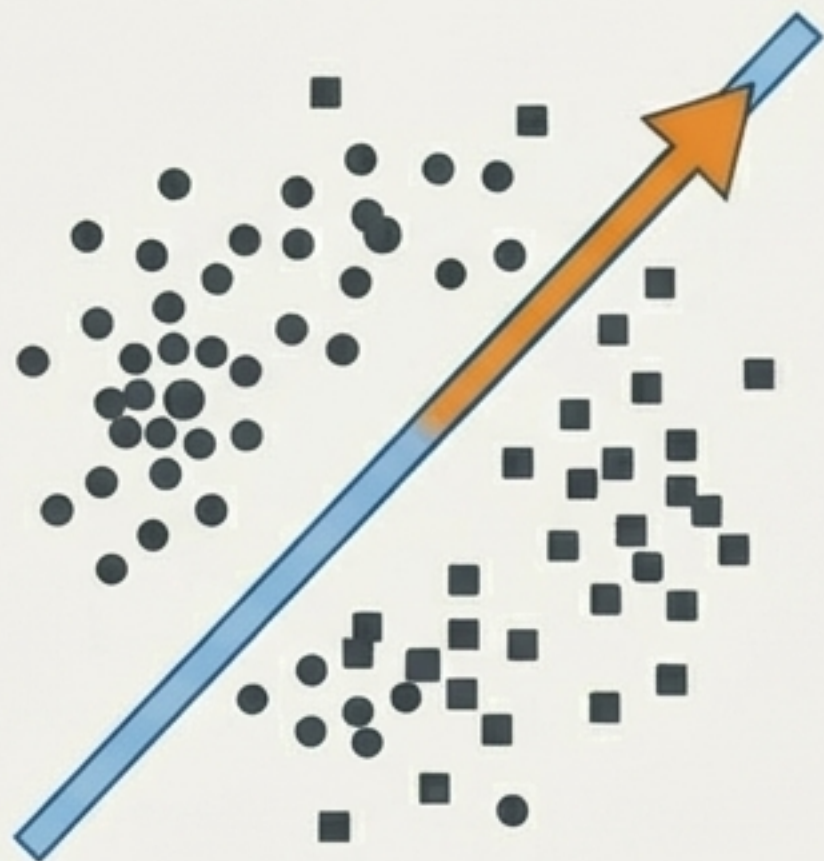
The Accelerator: `n_estimators`

This dial controls the maximum number of base models (stumps) trained sequentially.



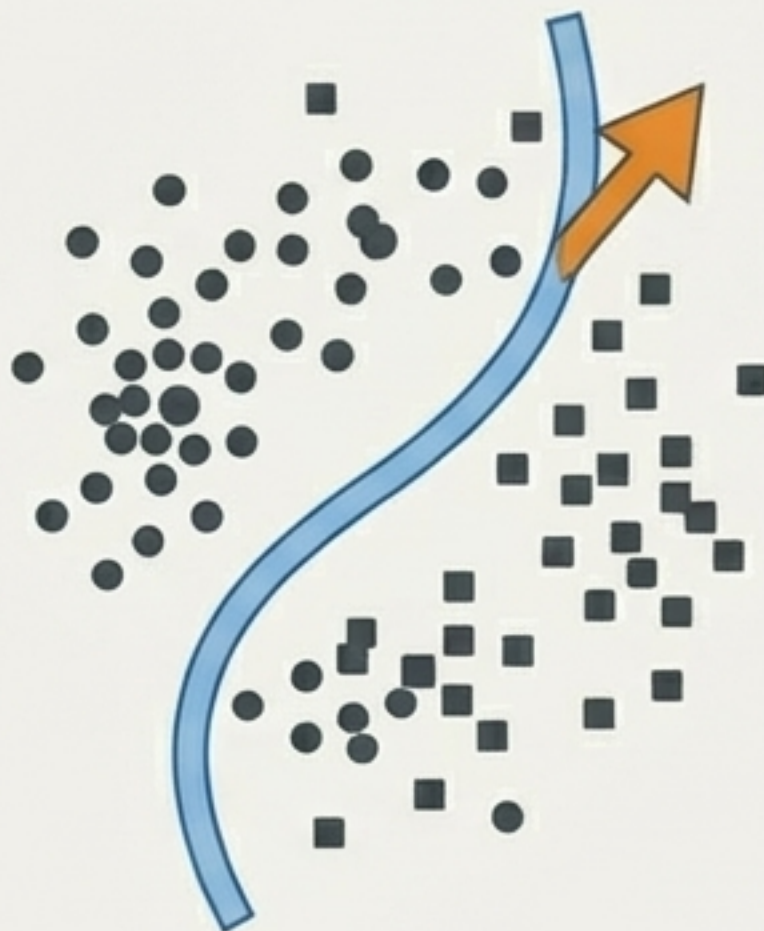
The Overfitting Spectrum

n=1 (Underfit)



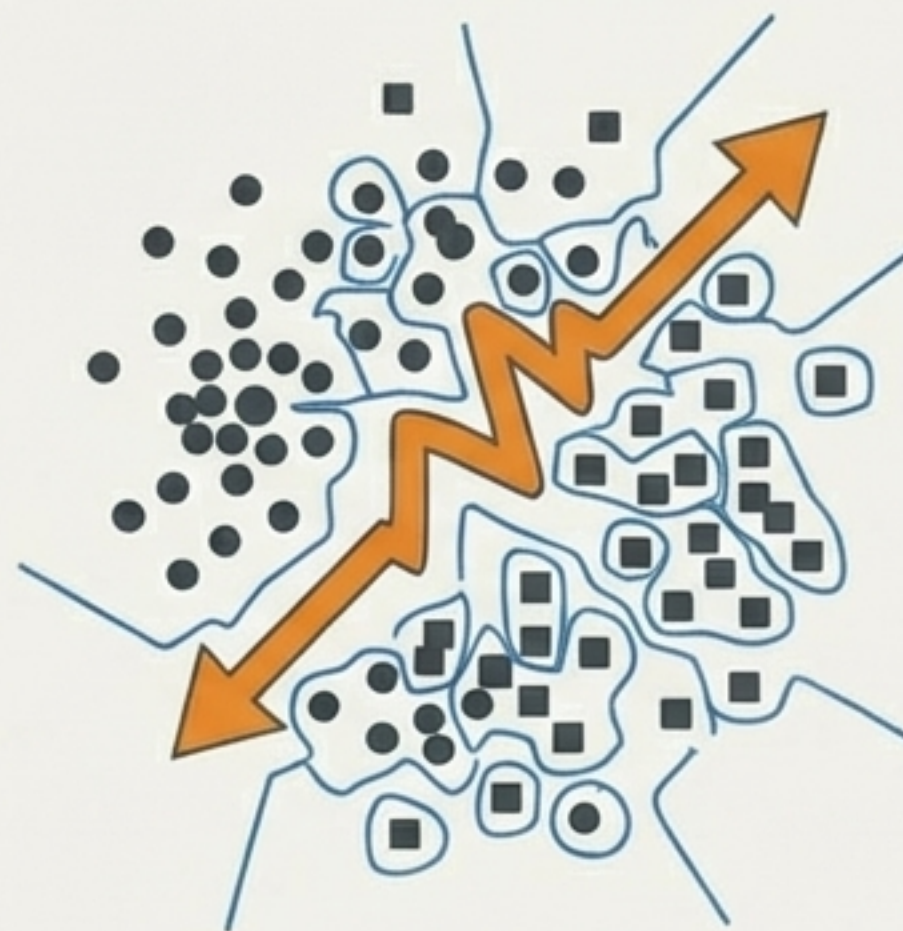
Way too simplistic.
Fails to capture
underlying patterns.

n=50 (Optimal)



Default baseline.
Strikes the ideal
balance of complexity.

n=1500 (Overfit)

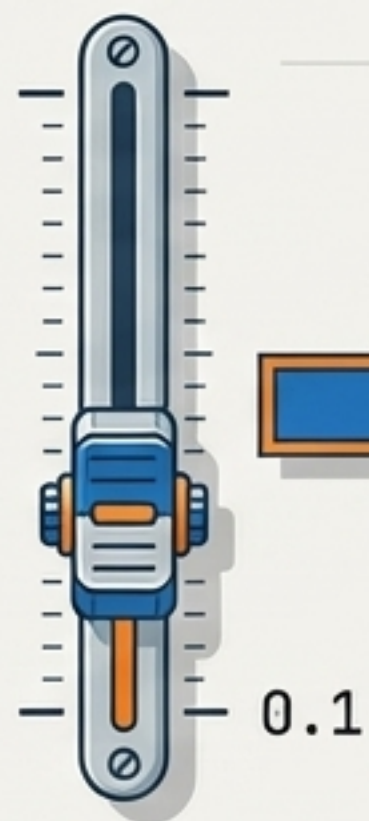


Catastrophic overfitting
& exceptionally high
training time.

The Brake: learning_rate

By default, the learning rate is 1.0, leaving Alpha untouched. When manually reduced (e.g., to 0.1), it acts as a mechanical brake, forcibly shrinking the Alpha value and reducing the influence of the current decision stump on the overall sequence.

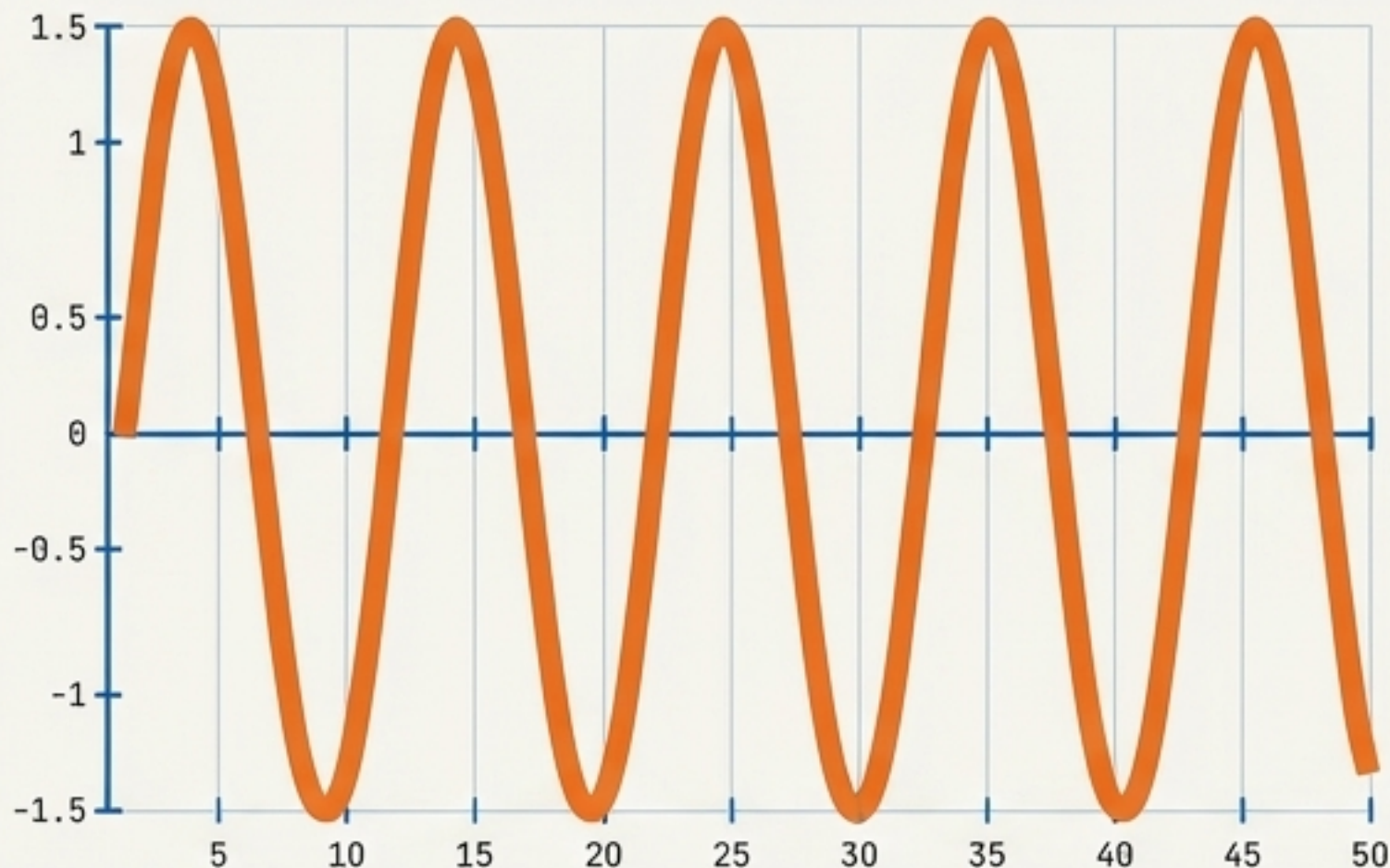
learning_rate



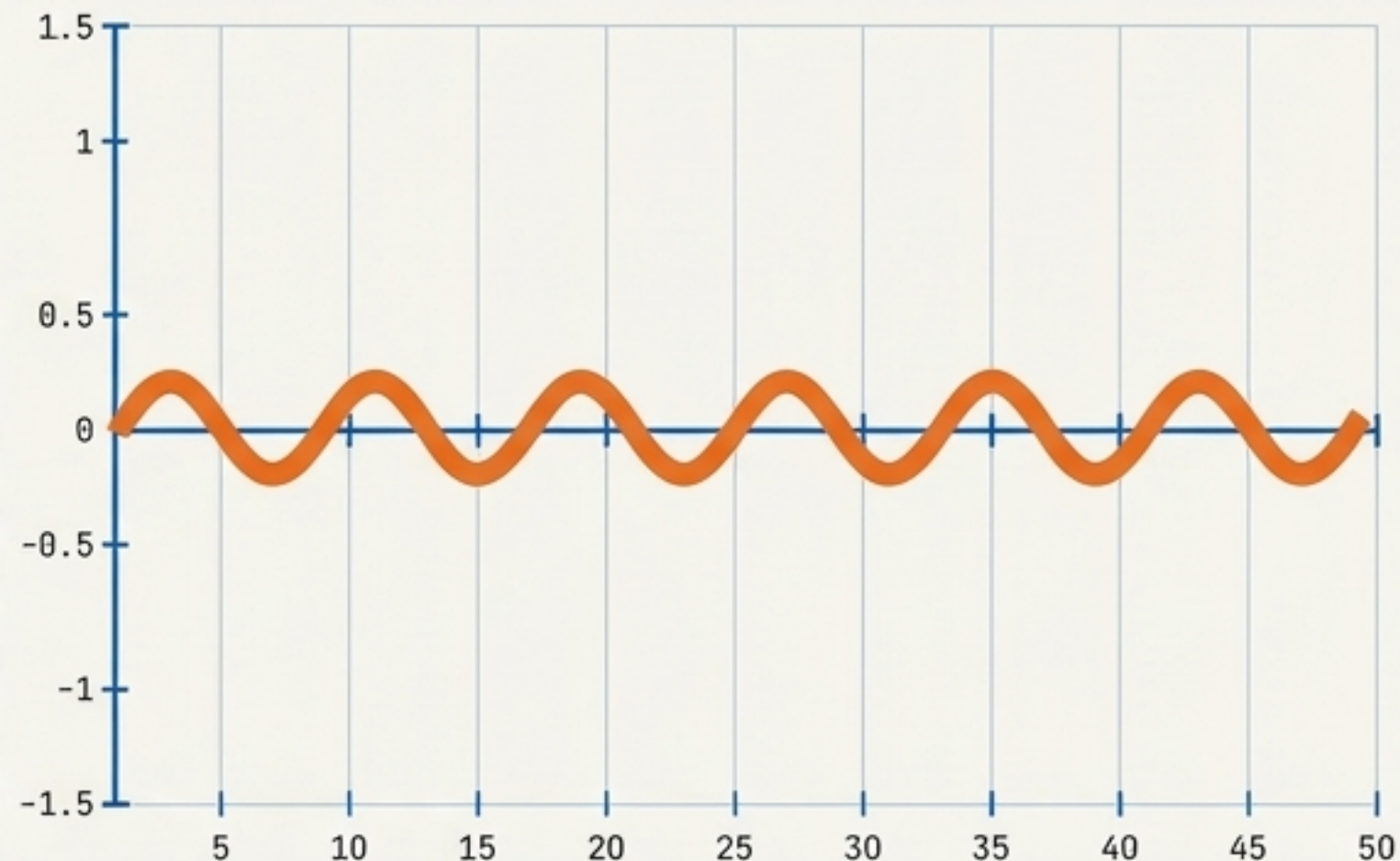
$$\text{Alpha} = 0.5 * \ln \left(\frac{1 - \text{error}}{\text{error}} \right)$$

Modulating the Amplitude of Weight Updates

Learning Rate = 1.0



Learning Rate = 0.1



Shrinkage Mechanics

A smaller Alpha directly reduces the amplitude of sample weight updates at each sequential step.

The Result

Smaller variations in weighted data points generate fewer extreme differences between weak boundaries. Learning is mathematically forced to slow down.

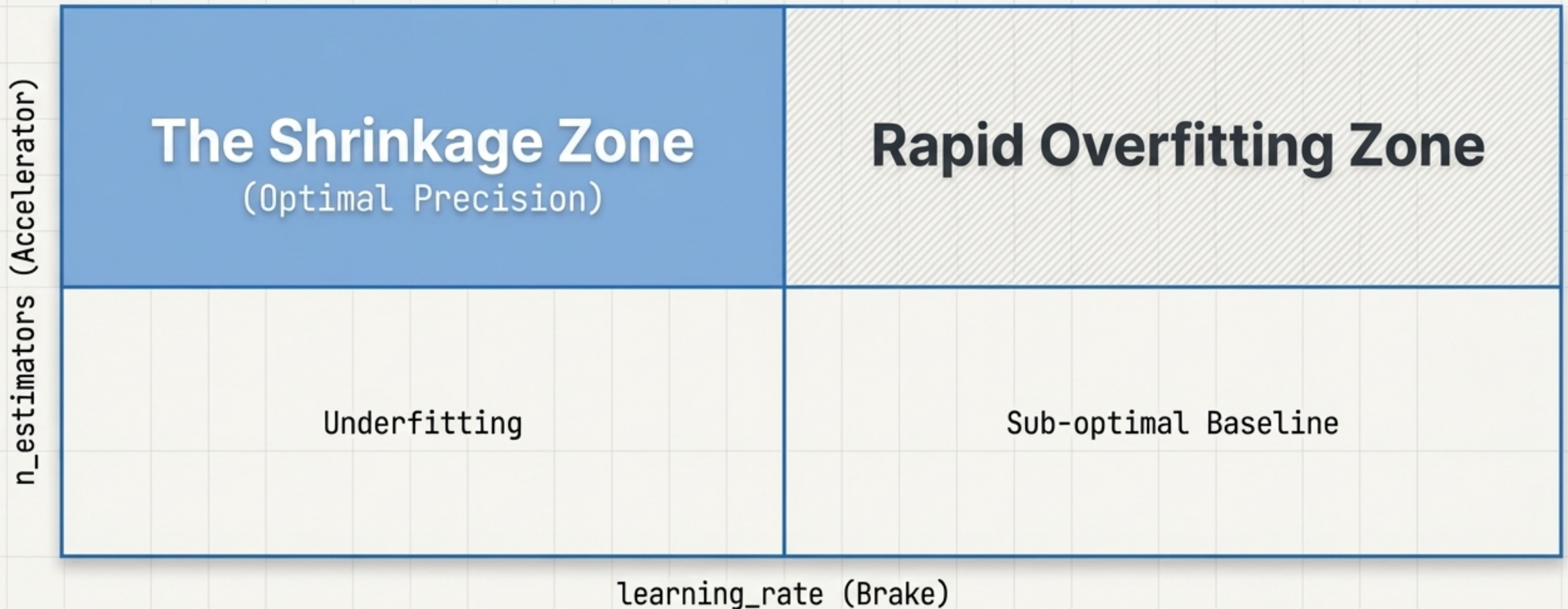
Algorithm Selection: SAMME vs. SAMME.R

The choice of boosting algorithm dictates the probability calculation method. SAMME.R provides a superior default baseline, actively achieving lower test error rates with significantly fewer iterations.

	SAMME	SAMME.R
Status	Legacy Parameter	Modern Default
Convergence	Slower	Faster
Test Error Rate	Higher	Lower
Iteration Efficiency	Requires more stumps	Requires fewer stumps

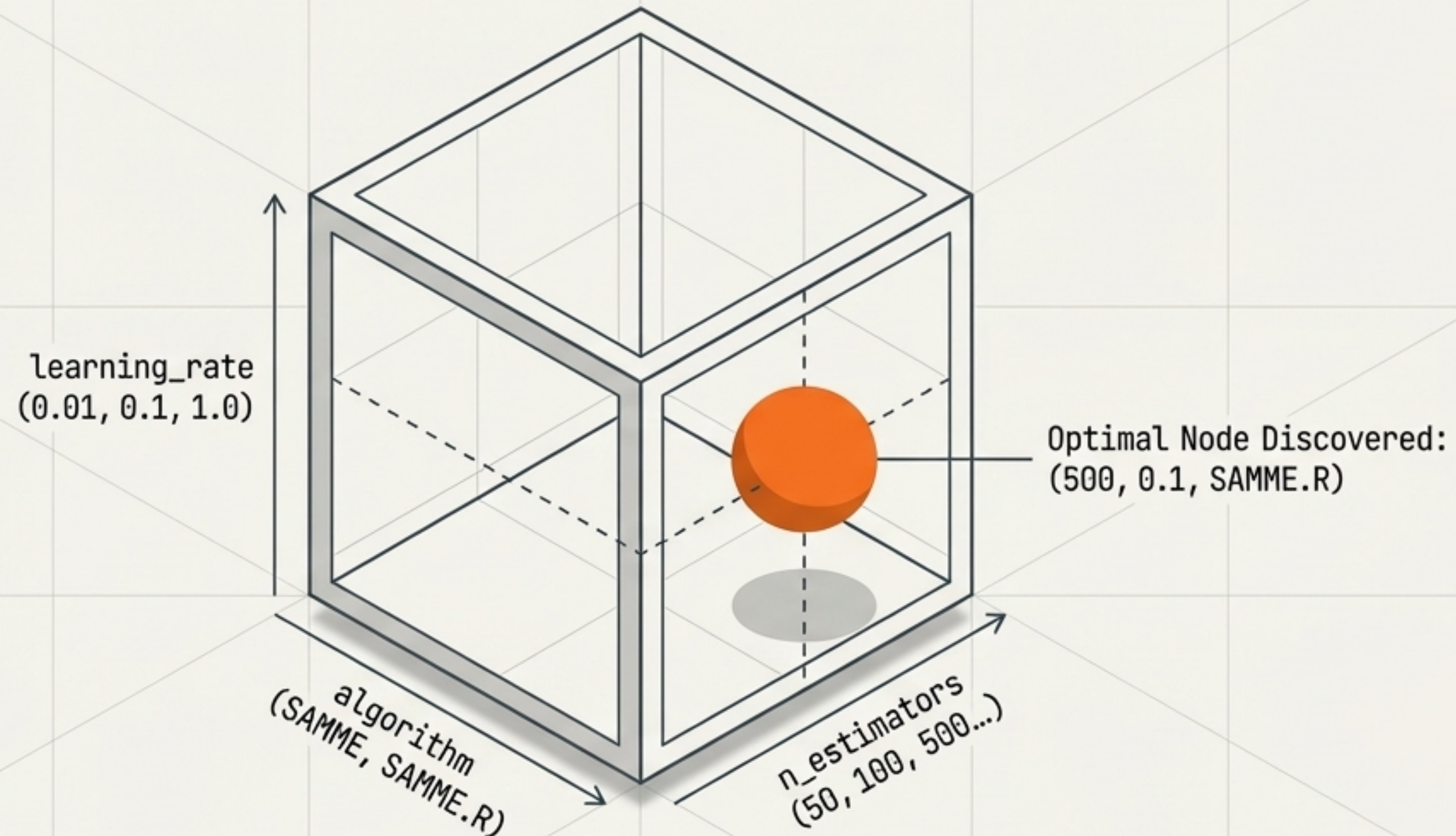
The Shrinkage Trade-off

The true power of AdaBoost is unlocked only when these two parameters are tuned in tandem. Increasing estimators inevitably triggers overfitting, but applying a low learning rate neutralises this effect. Hundreds of weak learners contribute tiny, highly precise adjustments, generating a smooth boundary.



Mapping the Hyperparameter Space

GridSearchCCV systematically navigates the multidimensional trade-off space. By defining a dictionary of constraints, the algorithm cross-validates every possible combination to locate the absolute mathematical peak of accuracy.



The Optimisation Pipeline

Implementing the search requires wrapping the base classifier in the GridSearchCV object. Setting `n_jobs=-1` is critical to offset the computational cost of testing thousands of model states.

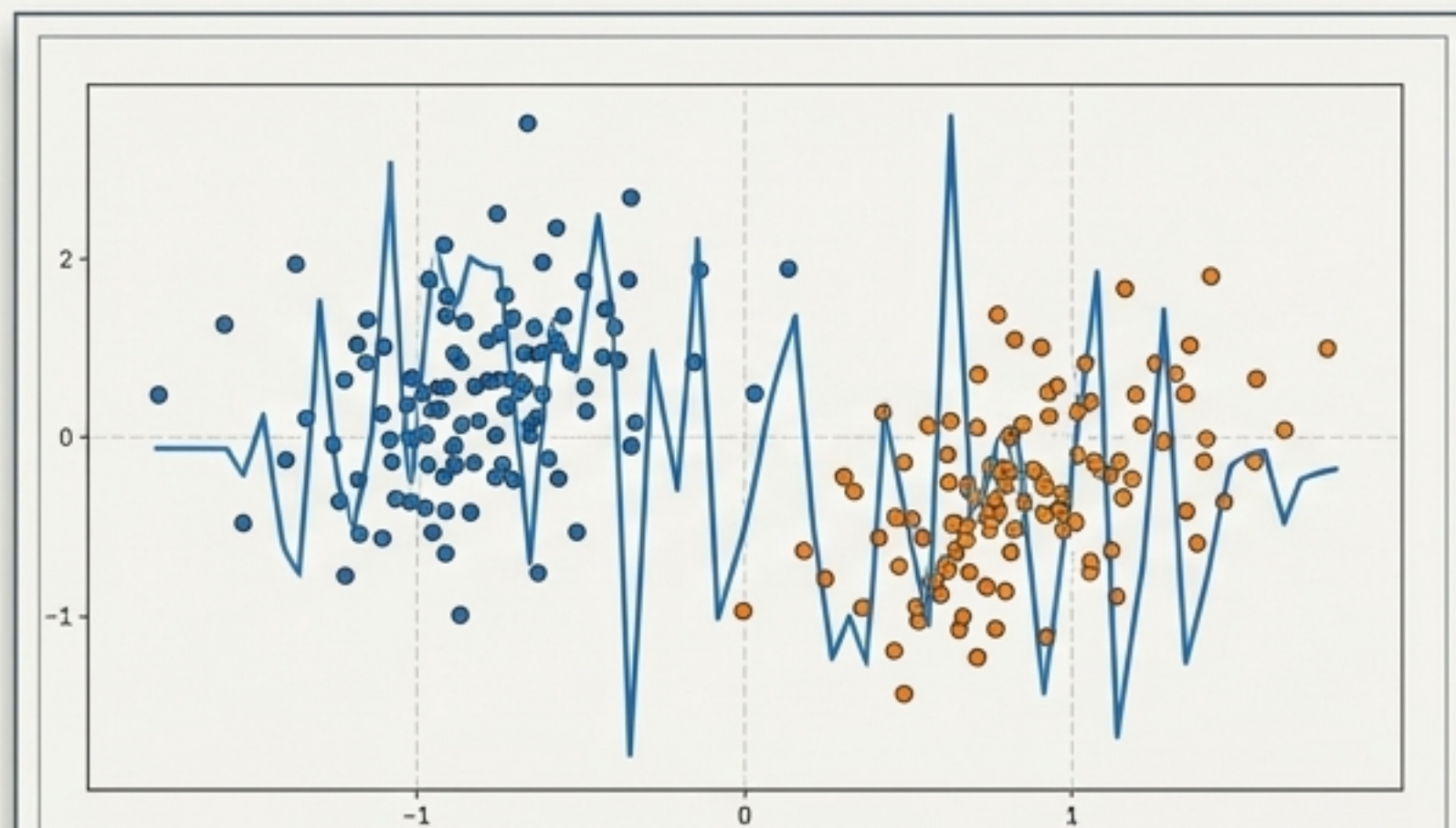
```
param_grid = {  
    'n_estimators': [50, 100, 500],  
    'learning_rate': [0.01, 0.1, 1.0],  
    'algorithm': ['SAMME', 'SAMME.R']  
}  
  
grid = GridSearchCV(  
    estimator=AdaBoostClassifier(),  
    param_grid=param_grid,  
    cv=10,   
    n_jobs=-1  
)
```

Robust 10-fold
cross-validation

Maximises processing speed
via parallel execution

The Yield: A 5% Precision Gain

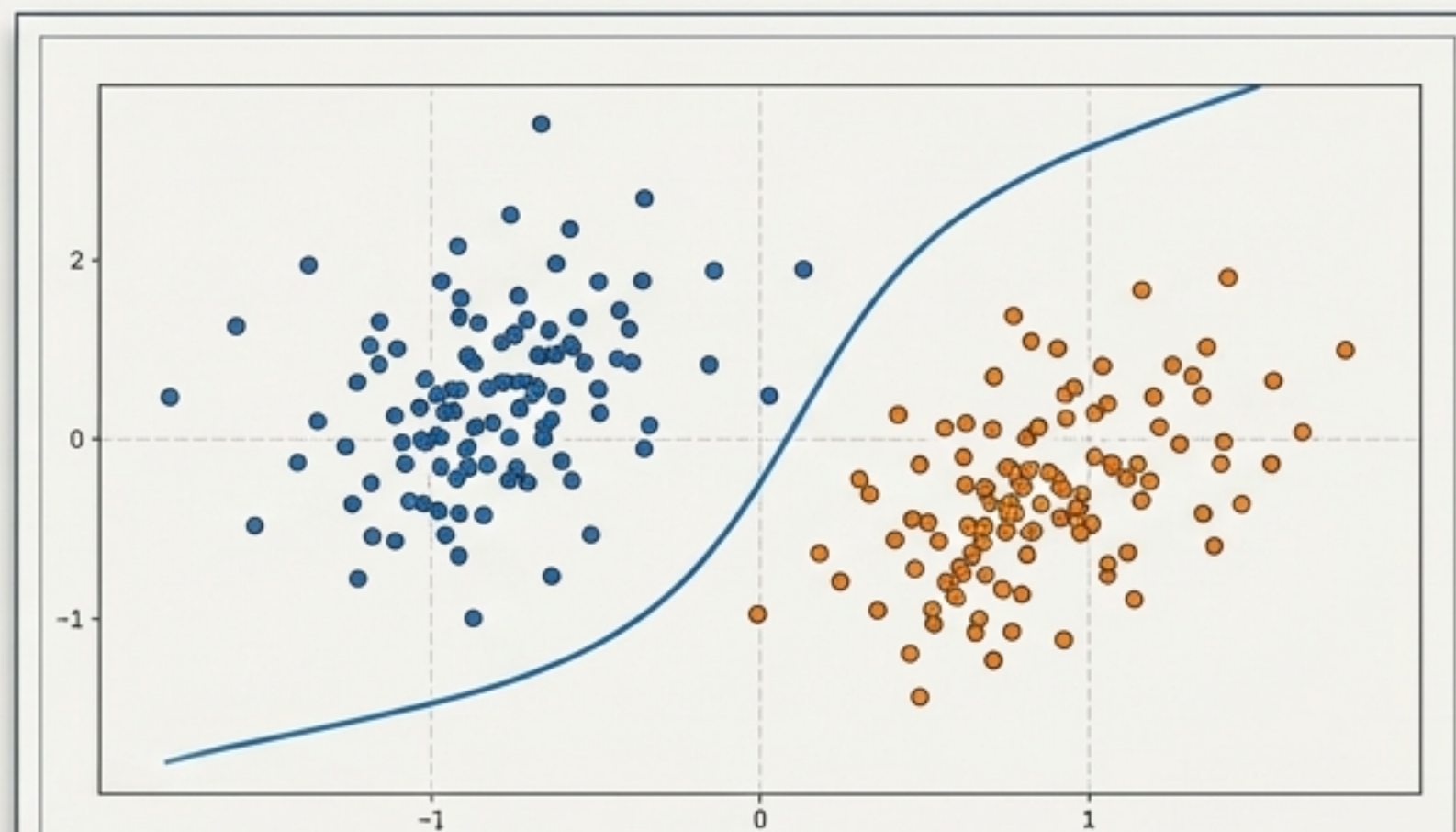
By embracing the Shrinkage trade-off—deploying 10x the number of estimators but reducing their individual amplitude by 90%—the algorithm escapes localised overfitting and discovers the optimal decision boundary.



Default State

$n=50$, $lr=1.0$

78% Accuracy



Tuned State

$n=500$, $lr=0.1$, $alg=SAMME.R$

83% Accuracy